

# Go – Cours 2

## Concurrence, subcommmands, gestion des erreurs

Et manipulation de données en JSON si on avance bien !



# Programme du jour

## Cours

---

Part 1

Rappels du cours précédent

Cours sur les thèmes du jour

## Live Coding

---

Part 2

Projet Gowatcher ensemble en cours

## TP

---

Part 3

Consignes et attentes

À vous de jouer

## Q&A

---

Part 4

# Cours



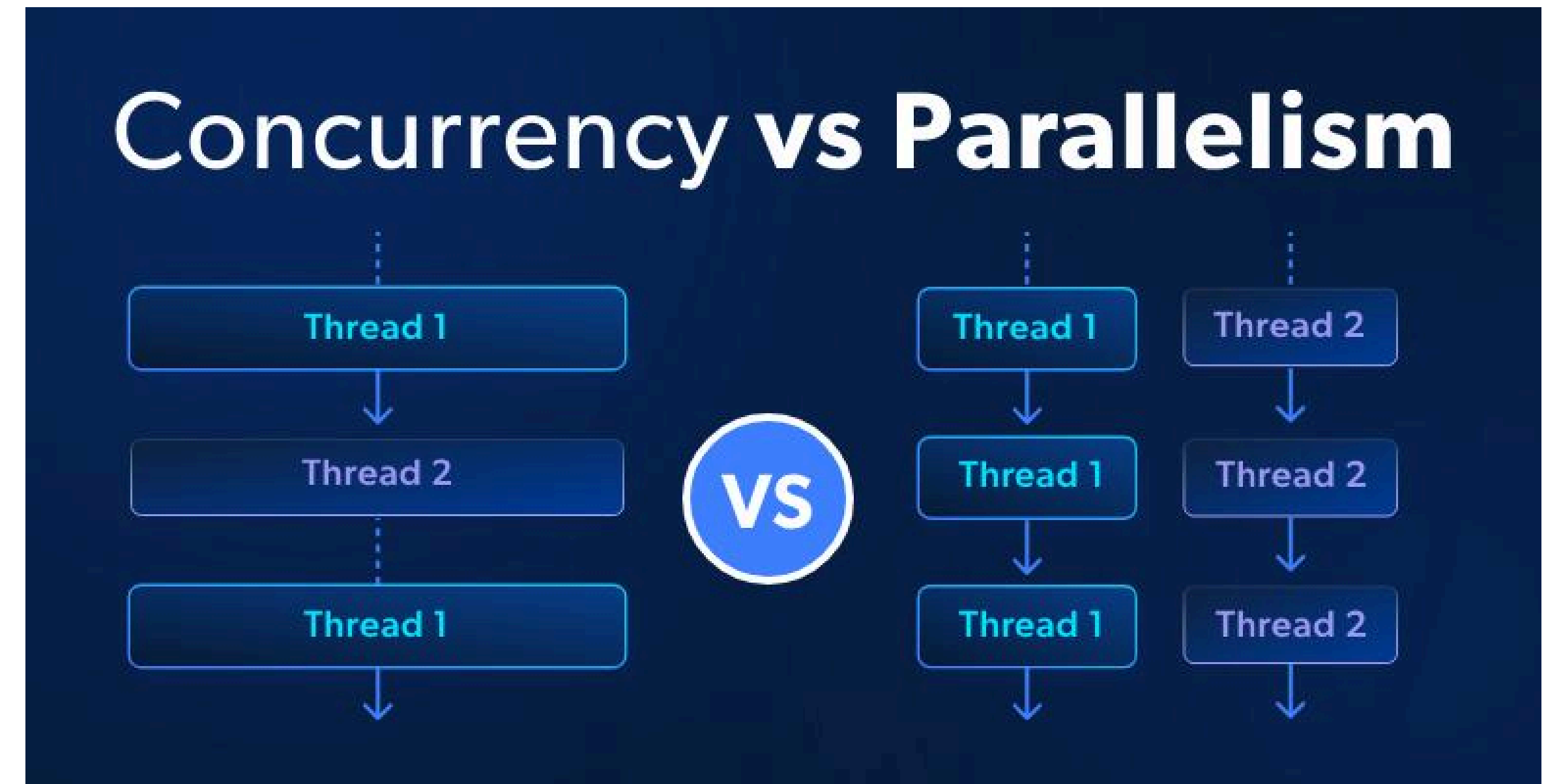
# La Concurrency



# Qu'est ce que la concurrence ?

## Concurrence ≠ Parallélisme

- **Concurrence** : plusieurs tâches gérées en même temps
- **Parallélisme** : plusieurs tâches exécutées en même temps
- Go : langage pensé pour la concurrence
  - Runtime optimisé
  - Primitives simples et efficaces



# Les Goroutines

## Exécution concurrente légère

- Une goroutine = une fonction lancée avec go
- Ultra-légères (quelques Ko)
- Des milliers possibles en parallèle

# Les Goroutines

## Exemple

```
func sayHello() {  
    fmt.Println("Hello !")  
}  
  
func main() {  
    go sayHello()  
    time.Sleep(1 * time.Second)  
}
```

🕒 Sans `Sleep`, la goroutine peut ne pas s'exécuter à temps

# Les channels

## Communication entre goroutines

- `chan type` : channel typé
- Envoi : `ch ← valeur`
- Réception : `val := ←ch`

```
ch := make(chan string)
go func() {
    ch <- "Hello"
}()
fmt.Println(<-ch)
```

💡 On utilise `make` pour créer un nouveau channel (comme avec les maps etc.)

# Propriétés des Channels

- Bloquants par défaut : synchronisent l'envoi et la réception
- Channels bufferisés : `make(chan int, 2)` pour éviter le blocage immédiat
- `select` : permet d'écouter plusieurs channels

# WaitGroup

## Synchroniser plusieurs goroutines

- `Add(n)` : indique combien de goroutines attendre
- `Done()` : appelée à la fin de chaque goroutine
- `Wait()` : attend que toutes soient terminées

```
wg.Add(1)
go func() {
    defer wg.Done()
    travail()
}()
wg.Wait()
```

💡 `defer` permet de programmer une fonction pour qu'elle s'exécute automatiquement à la fin de la fonction courante, même en cas d'erreur ou de retour anticipé.

# select : Multiplexage de Channels

## Attente sur plusieurs channels

```
select {  
  case msg := <-ch1:  
    fmt.Println("Ch1 :", msg)  
  case msg := <-ch2:  
    fmt.Println("Ch2 :", msg)  
  default:  
    fmt.Println("Aucun message")  
}
```

💡 Même syntaxe que le `switch`

# Mutex et sections critiques

## Accès sécurisé aux données partagées

```
var mu sync.Mutex
mu.Lock()
// section critique
mu.Unlock()
```

 Préférer les channels pour la synchronisation quand c'est possible

# Résumé des concepts de base

| Concept          | Rôle                                 |
|------------------|--------------------------------------|
| <u>Goroutine</u> | Fonction concurrente légère          |
| Channel          | Communication entre tâches           |
| WaitGroup        | Synchronisation de <u>goroutines</u> |
| select           | Écoute multiple de channels          |
| Mutex            | Protection d'accès aux données       |
|                  |                                      |

# Subcommands & libraries CLI



# Les subcommands

## Qu'est-ce qu'une subcommand ?

- Commande secondaire dans une CLI
- Exemple : `git clone`, `git push`, `docker run`
- Permet d'organiser plusieurs actions dans un seul outil

# Pourquoi on les utilise ?

- Structure logique et claire pour outils complexes
- Une action = une subcommand
- Améliore lisibilité et usage (UX en ligne de commande)

# Subcommands & flags

| Élément       | Rôle                        | Exemple                                        |
|---------------|-----------------------------|------------------------------------------------|
| Flag          | Option modifiant une action | <code>--depth=1</code>                         |
| <u>Subcmd</u> | Action à part entière       | <code>git clone</code> , <code>git push</code> |

# Package GO pour CLI

| Package                                    | Description                                                                                                                                                     | Usage typique                         |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| <code>flag</code>                          | Package standard pour parsing de flags simples                                                                                                                  | Projets basiques, flags simples       |
| <code>os.Args</code>                       | Lecture brute des arguments                                                                                                                                     | Parsing manuel                        |
| <code>github.com/spf13/cobra</code>        | Bibliothèque puissante pour CLI avec subcommands, flags, autocompletion                                                                                         | CLI complexes, projets professionnels |
| <code>github.com/urfave/cli</code>         | Autre framework populaire, facile à utiliser                                                                                                                    | CLI simples à complexes               |
| <code>github.com/alecthomas/kingpin</code> | Framework pour CLI avec validation et subcommands                                                                                                               | CLI avancés, besoin de validation     |
| <code>github.com/google/subcommands</code> | <b>Approche basée sur des interfaces.</b> Structuré et orienté objet. Idéal pour certains cas d'usage où l'on veut structurer les commandes en types distincts. | CLI simples à complexes               |

# Pourquoi Cobra ?

- Structure claire : commandes, sous-commandes, flags
- Génération automatique de l'aide & auto-completion
- Support flags locaux et globaux
- Intégration facile avec Viper pour config
- Utilisé par : Kubernetes, Hugo, Docker Compose, etc.

# Structure d'un projet Cobra

```
your-cli-app/  
├─ main.go  
└─ cmd/  
    ├─ root.go      // La commande principale (souvent sans nom)  
    ├─ serve.go     // Ex: myapp serve  
    ├─ create.go    // Ex: myapp create user  
    └─ ...
```

# Installation de Cobra

```
go get github.com/spf13/cobra/cobra
```

# Exemple

```
package cmd

import (
    "fmt"

    "github.com/spf13/cobra"
)

// createCmd représente la commande 'create'
var createCmd = &cobra.Command{
    Use:     "create",
    Short:   "Crée une nouvelle ressource (utilisateur, article, etc.)",
    Long:    `La commande 'create' permet de créer différentes ressources.
    Exemples:
    myapp create user --name John
    myapp create article --title "My Post"`,
    Run: func(cmd *cobra.Command, args []string) {
        fmt.Println("create called")
    },
}

func init() {
    createCmd.AddCommand(createUserCmd) // Attache 'createUserCmd' à 'createCmd'

    // Définition des flags pour la sous-commande 'user'
    createUserCmd.Flags().StringVarP(&createUserName, "name", "n", "", "Nom de l'utilisateur")
    createUserCmd.Flags().StringVarP(&userEmail, "email", "e", "", "Email de l'utilisateur")
    createUserCmd.MarkFlagRequired("name") // Rend le flag 'name' obligatoire
}
```

# Gestion des erreurs



# Les erreurs en Go

- Pas de try/catch : Les erreurs sont des valeurs retournées
- nil = Pas d'erreur

## Avantages :

- **Clarté** : Les erreurs sont explicites dans le code.
- **Simplicité** : Pas de système d'exception complexe.
- **Performance** : Pas de surcharge liée à la gestion des exceptions.
- **Contrôle total** : Le développeur décide exactement comment gérer chaque erreur.

# Les erreurs en Go

## Les inconvénients :

- **Verbo­sité** : Beaucoup de `if err != nil` répétitifs.
- **Pas de propagation automatique** : Il faut relayer les erreurs manuellement.
- **Peu de typage des erreurs**

# Le type `error` en Go

```
type error interface {  
    Error() string  
}
```

- Toute structure ou valeur qui implémente la méthode `Error()` peut être utilisée comme erreur

# Gestion des erreurs

```
func divide(a, b float64) (float64, error) {  
    if b == 0 {  
        return 0, errors.New("division by zero")  
    }  
    return a / b, nil  
}
```

Usage :

```
|result, err := divide(10, 0)  
if err != nil {  
    fmt.Printf("Erreur :%s\n", err)  
} else {  
    fmt.Println("Résultat :", result)  
}
```

# Helpers

```
errors.new(text string) error
```

type d'erreur est suffisant pour les cas simples mais limité : pas de contexte, pas de type spécifique.

```
return fmt.Errorf("cannot read file %s: %w", filename, err)
```

Le `%w` permet de **wrapper** une erreur tout en ajoutant du contexte. Crucial pour le débogage. Compatible avec `errors.Is()` et `errors.As()`

# errors.Is(): Tester une erreur spécifique

- Vérifie si une erreur correspond à une erreur cible, directement ou via un wrapping (%w)
- Pourquoi l'utiliser ?
  - Pour savoir si une erreur particulière (ex. io.EOF, os.ErrNotExist) s'est produite.
  - Fonctionne même si l'erreur a été enrichie avec fmt.Errorf(..., %w).

# errors.Is(): Exemple d'utilisation

```
var ErrInvalid = errors.New("invalid input")

func do() error {
    return fmt.Errorf("validation failed: %w", ErrInvalid)
}

err := do()

if errors.Is(err, ErrInvalid) {
    fmt.Println("Entrée invalide détectée")
}
```

# errors.As() : récupérer une erreur par type

- Vérifie si une erreur contient un type spécifique, et permet d'y accéder.
- Pourquoi l'utiliser ?
  - Pour extraire des informations spécifiques d'une erreur personnalisée (struct).
  - Permet de traiter différents types d'erreurs différemment.

# Les erreurs personnalisées

```
type MyError struct {  
    Code int  
    Msg  string  
}  
  
func (e MyError) Error() string {  
    return fmt.Sprintf("Erreur [%d]: %s", e.Code, e.Msg)  
}
```

Utilisation :

```
return 0, MyError{Code: 400, Msg: "Invalid input"}
```

# Les erreurs personnalisées

```
type MyError struct {  
    Code int  
    Msg  string  
}  
  
func (e MyError) Error() string {  
    return e.Msg  
}  
  
func do() error {  
    return MyError{Code: 403, Msg: "unauthorized"}  
}  
  
err := do()  
  
var e MyError  
if errors.As(err, &e) {  
    fmt.Println("Code erreur :", e.Code)  
}
```

**As** permet de "caster" une erreur vers un **type concret**.

# Manipulation de données en JSON



# Package bibliothèque standard

```
import "encoding/json"
```

- Le package standard encoding/json fournit les fonctionnalités nécessaires pour encoder et décoder JSON.
- Pas besoin de librairies tierces pour les opérations de base.

## Fonctions principales :

- `json.Marshal()` : Convertit une structure Go en JSON (encodage/sérialisation).
- `json.Unmarshal()` : Convertit du JSON en une structure Go (décodage/désérialisation).
- `json.MarshalIndent()` : Sérialisation qui respecte l'indentation

# Exemple sérialisation/Marshalling

```
package main

import (
    "encoding/json"
    "fmt"
)

type Personne struct {
    Nom    string
    Age    int
    Ville  string
}

func main() {
    p := Personne{"Jean", 25, "Paris"}
    jsonData, err := json.Marshal(p)
    if err != nil {
        fmt.Println("Erreur lors de l'encodage:", err)
        return
    }
    fmt.Println(string(jsonData)) // Output: {"Nom":"Jean","Age":25,"Ville":"Paris"}
}
```

# Exemple désérialisation/Unmarshalling

```
package main

import (
    "encoding/json"
    "fmt"
)

type User struct {
    Username string
    Email    string
}

func main() {
    jsonData := []byte(`{"Username":"john_doe","Email":"john@example.com"}`)
    var user User
    err := json.Unmarshal(jsonData, &user)
    if err != nil {
        fmt.Println("Erreur lors du décodage:", err)
        return
    }
    fmt.Printf("Utilisateur: %+v\n", user)
}
```

# Les tags de structures

- Syntaxe : ``json:"nom_json,option"``
- `json:"nom_json"` : Renomme le champ JSON.
- `json:"-"` : Ignore le champ lors de l'encodage/décodage.
- `json:"omitempty"` : Omet le champ si sa valeur est la valeur zéro du type (0, "", nil, false).
- `json:",string"` : Encode/décode comme une chaîne (utile pour de grands entiers par exemple).

```
type Produit struct {  
    ID      string `json:"product_id"`  
    Nom     string `json:"name"`  
    Prix    float64 `json:"price,omitempty"`  
    Quantite int    `json:"- "` // Ignoré  
    Details string `json:"details,string"` // Encodé/décodé comme une chaîne  
}
```

# Les tags de structures

```
type Adresse struct {  
    Ville  string `json:"ville"`  
    Pays   string `json:"pays"`  
}  
  
type Utilisateur struct {  
    Nom      string `json:"nom"`  
    Adresse Adresse `json:"adresse"`  
}
```

# Lecture d'un fichier JSON

```
data, err := os.ReadFile("utilisateur.json")
if err != nil {
    log.Fatal(err)
}

var u Utilisateur
json.Unmarshal(data, &u)
```

# Écriture d'un fichier JSON

```
data, _ := json.MarshalIndent(u, "", " ")  
os.WriteFile("utilisateur.json", data, 0644)
```

# TP GoAnalyzer



# Q&A



# Slides Bonus



# defer en Go

- Utilisé pour retarder l'exécution d'une fonction jusqu'à la fin de la fonction appelante
- Très pratique pour fermer des ressources (fichiers, connexions) ou nettoyer
- Garantit que le code est exécuté même en cas d'erreur ou de return prématuré
- S'utilise souvent ainsi :

```
defer resource.Close()
```